Localization for Autonomous Navigation

Chapter 7. Kalman Filter Math

Dept. Intelligent Systems & Robots

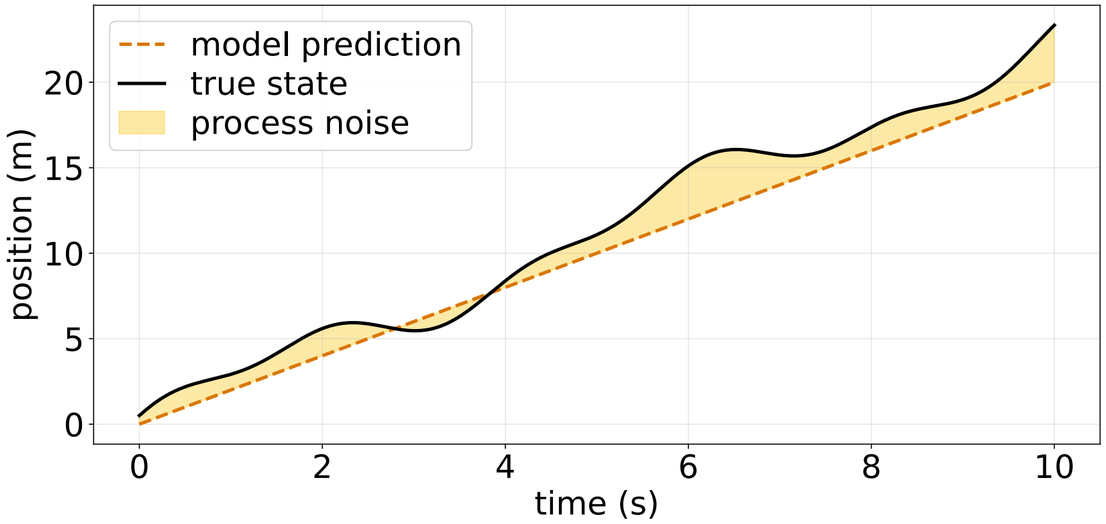
Prof. SungTae Moon

Contents

- Modeling a Dynamic System
- State-Space Representation of Dynamic Systems
- Design of the Process Noise Matrix
- Stable Computation of the Posterior Covariance
- Deriving the Kalman Gain Equation
- Numeric Integration of Differential Equations
- Bayesian Filtering
- Converting Kalman Filter to a g-h Filter

Modeling a Dynamic System

- **동적 시스템**: 위치, 온도 등 상태가 시간에 따라 변하는 물리 시스템, 미분방정식으로 모델링
- **Process**: 자동차는 일정 시간 동안 속도만큼 이동, 속도는 가속도에 따라 변화
- **현실**: 도로 요철, 바람 저항, 언덕, 서스펜션 마찰로 완벽한 모델링은 불가능
- **Process noise**: 모델이 설명하지 못하는 부분, 무작위 사건만 뜻하지 않음 (예: 진공을 가정한 공 모델의 공기 저항)



$$v = at, \quad x = \frac{1}{2}at^2 + v_0 t + x_0$$

$$v = \frac{dx}{dt}, \quad a = \frac{dv}{dt} = \frac{d^2x}{dt^2}$$

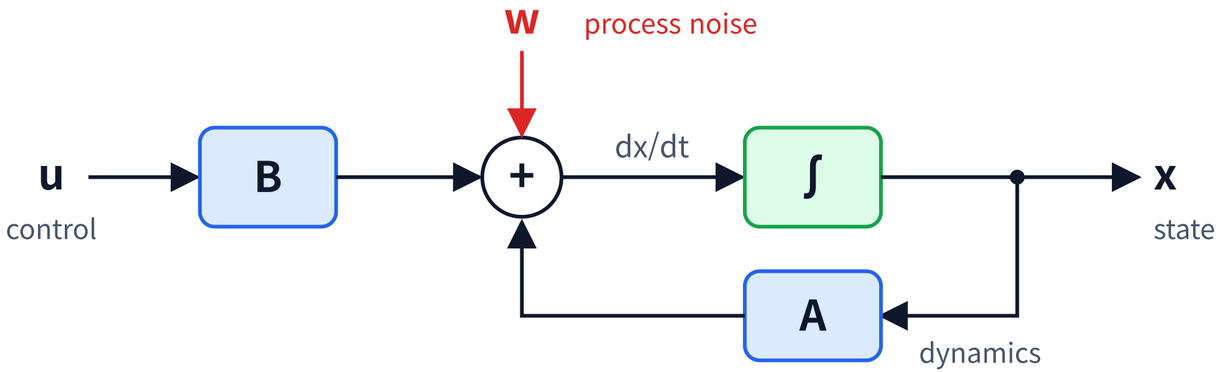
$$x(t) = x_{pred}(t) + noise(t)$$

> **핵심**: 참 상태는 불완전한 모델의 예측값에 알 수 없는 process noise 를 더한 값으로 표현

System Model with Noise and Control

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} \qquad \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{w} \qquad \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{w}$$

- **A**: system dynamics matrix, 고차 미분방정식을 1 차 미분방정식 집합으로 바꾼 뒤의 시스템 동특성
- **w**: process noise, Kalman filter 는 white noise 를 가정
- **u, B**: 제어 입력과 그 영향을 상태 변화로 바꾸는 행렬 (예: 가속 페달, 공에 작용하는 중력)



> **핵심**: Kalman 이 w 의 특정 성질을 가정하고 최적 추정기를 찾은 식, 이후 모든 설계의 출발점

From Derivative to Recursion

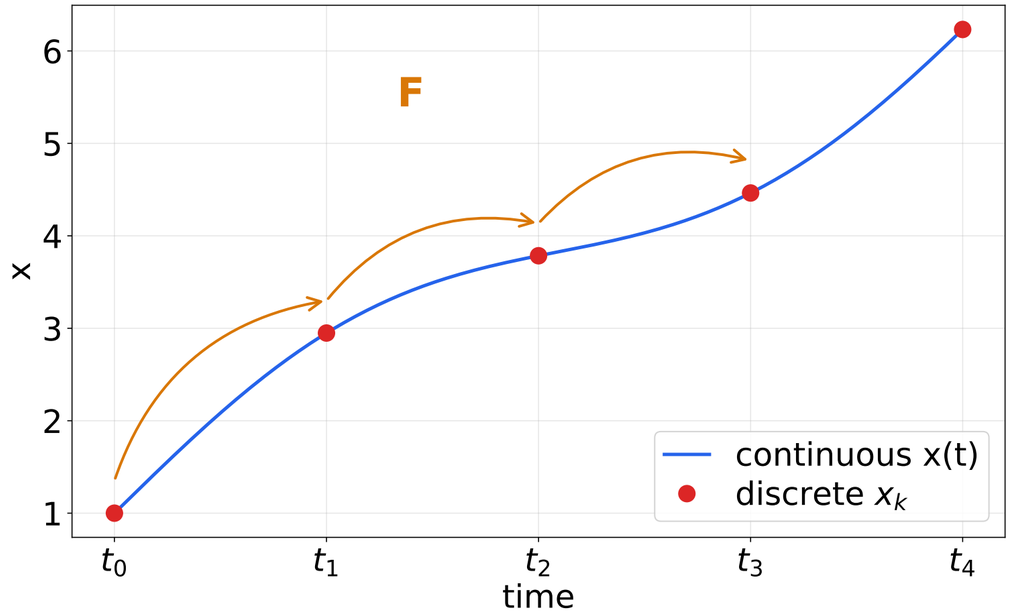
- **목표**: 미분이 아닌 x 자체, x_{k-1} 로 x_k 를 재귀적으로 계산
- **F**: state transition matrix, Δt 동안의 이산 선형 전이

$$\mathbf{x}(t_k) = \mathbf{F}(\Delta t)\mathbf{x}(t_{k-1}) + \mathbf{B}(t_k)\mathbf{u}(t_k)$$

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}_k\mathbf{u}_k$$

$$\int_{x_{k-1}}^{x_k} dx = \int_0^{\Delta t} v\, dt \quad \Rightarrow \quad x_k = x_{k-1} + v\Delta t$$

$$\mathbf{x}_k = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_{k-1} \\ \dot{x}_{k-1} \end{bmatrix}$$



한계: 가장 단순한 미분방정식 $\dot{x} = v$ 이기에 가능, 대부분의 물리 시스템은 이 방식으로 풀리지 않음

State-Space Methods

- **등장**: Apollo 계획 무렵 Kalman 의 연구로 널리 사용
- **절차**: n 차 미분방정식을 등가인 1 차 미분방정식 집합으로 변환한 뒤 벡터-행렬 형태로 정리
- **이산화**: 여러 기법으로 선형 미분방정식을 재귀식으로 변환
- **명칭**: 해를 시스템 상태로 표현하므로 state-space, F 를 fundamental matrix 또는 Φ 로 표기하기도 함

n-th order ODE

$a_n y^{(n)} + \ldots + a_0 y = u$

→

First-order ODEs

$dx_1/dt = x_2, \ldots$

→

Vector-matrix form

$dx/dt = Ax + Bu$

→

Recursive form

$x_k = F x_{k-1} + B u_k$

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad \Longrightarrow \quad \mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}_k\mathbf{u}_k$$

> **핵심**: 연속 모델 A 에서 이산 모델 F 를 구하는 것이 state-space 방법의 목표

Higher Order to First Order

$$a_n \frac{d^n y}{dt^n} + a_{n-1} \frac{d^{n-1} y}{dt^{n-1}} + \cdots + a_2 \frac{d^2 y}{dt^2} + a_1 \frac{dy}{dt} + a_0 y = u$$

- **방법**: 미분 항마다 새 변수를 정의해 고차 방정식을 1 차 방정식 집합으로 축소, 예: $\ddot{x} - 6\dot{x} + 9x = u$

1. Isolate

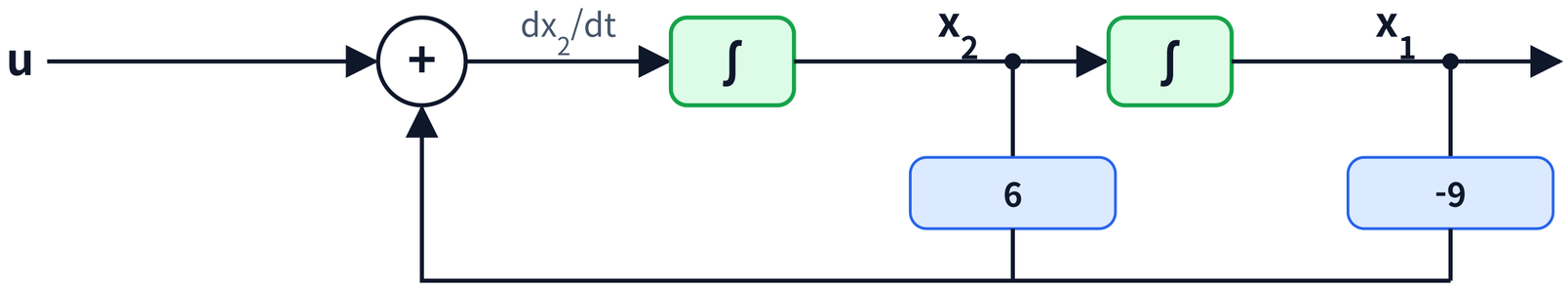
$$\ddot{x} = 6\dot{x} - 9x + u$$

2. Define

$$x_1 = x, \quad x_2 = \dot{x}$$

3. Substitute

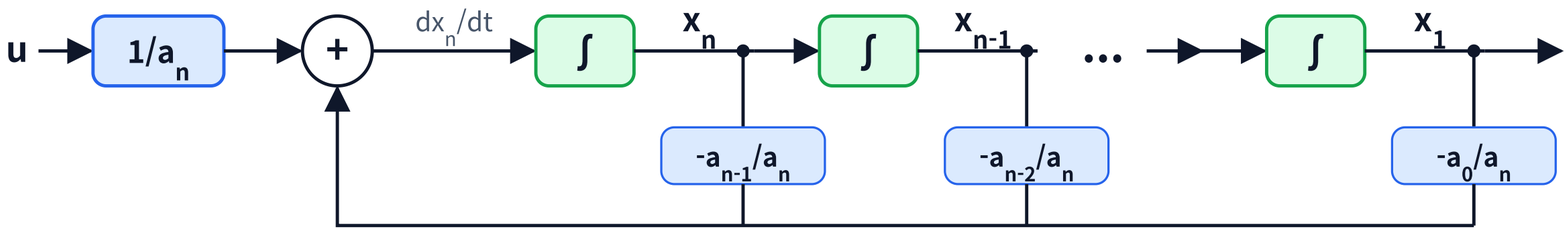
$$\dot{x}_1 = x_2$$
$$\dot{x}_2 = 6x_2 - 9x_1 + u$$



State-Space Form

$$\frac{dx_1}{dt} = x_2,\ \frac{dx_2}{dt} = x_3,\ \ldots,\ \frac{dx_{n-1}}{dt} = x_n \qquad \frac{dx_n}{dt} = -\frac{1}{a_n}\sum_{i=0}^{n-1} a_i x_{i+1} + \frac{1}{a_n}u$$

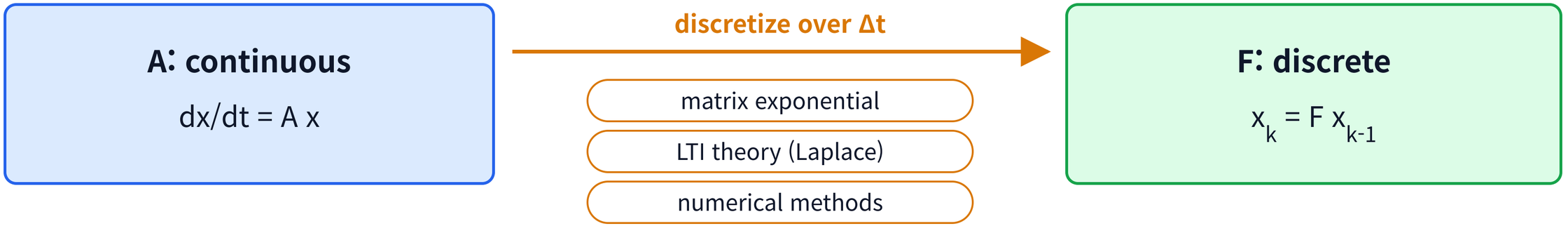
$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \vdots \\ \dot{x}_n \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -\frac{a_0}{a_n} & -\frac{a_1}{a_n} & -\frac{a_2}{a_n} & \cdots & -\frac{a_{n-1}}{a_n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ \frac{1}{a_n} \end{bmatrix} u \quad = \mathbf{Ax} + \mathbf{Bu}$$



Finding the Fundamental Matrix

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} \qquad \mathbf{x}(t_k) = \mathbf{F}(\Delta t)\mathbf{x}(t_{k-1}) \qquad \mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1}$$

- **A**: 연속 미분방정식 집합, system dynamics matrix
- **F**: Δt 동안 상태를 전파하는 이산 선형 방정식 집합
- **표기**: 관례상 t_k 와 (Δt) 를 생략해 간단히 표기
- **방법**: matrix exponential (가장 흔함), LTI 이론, 수치 기법



> **핵심**: Kalman filter 문헌과 실무에서 만나는 F 계산법은 대부분 이 세 가지 중 하나

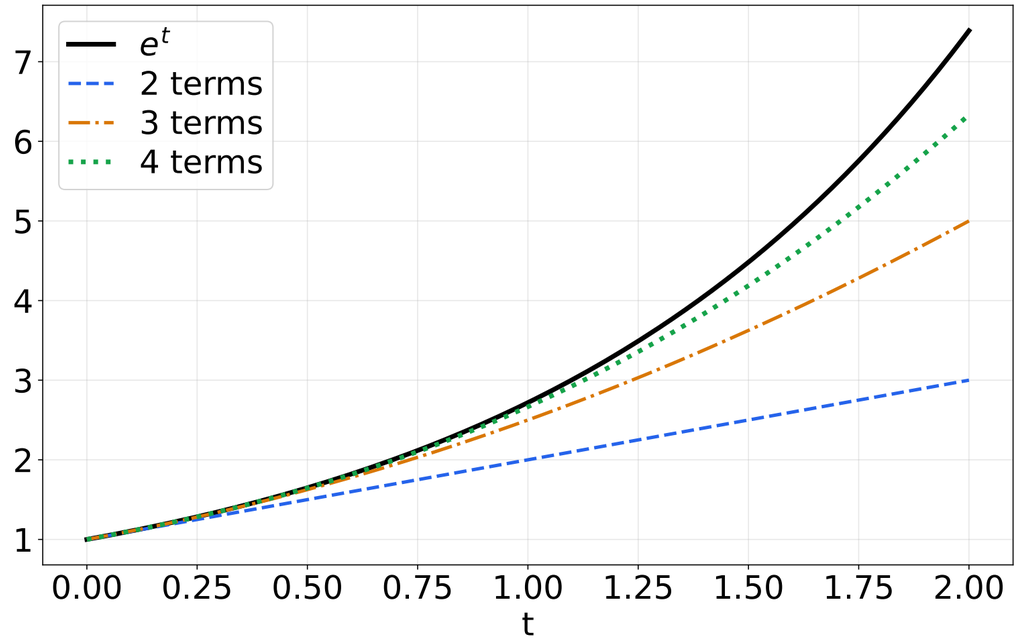
Matrix Exponential

Scalar

$$\frac{dx}{dt} = kx$$
$$\frac{dx}{x} = k\,dt$$
$$\int \frac{1}{x}dx = \int k\,dt$$
$$\log x = kt + c$$
$$x = e^c e^{kt} = c_0 e^{kt}$$

Matrix

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x},\ \mathbf{x}(0) = \mathbf{x}_0$$
$$\mathbf{x} = e^{\mathbf{A}t}\mathbf{x}_0$$
$$\mathbf{F} = e^{\mathbf{A}\Delta t}$$
$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1}$$



핵심: F 를 찾는 문제가 $e^{At} = I + At + (At)^2/2! + \cdots$ 계산으로 축소, 항을 늘릴수록 정확

Newton's Equations

$$\begin{bmatrix}\dot{x}\\ \dot{v}\end{bmatrix} = \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\begin{bmatrix}x\\ v\end{bmatrix}, \quad \mathbf{A}^2 = \begin{bmatrix}0 & 0\\ 0 & 0\end{bmatrix}$$

$$\mathbf{F} = e^{\mathbf{A}\Delta t} = \mathbf{I} + \mathbf{A}\Delta t + \frac{(\mathbf{A}\Delta t)^2}{2!} + \frac{(\mathbf{A}\Delta t)^3}{3!} + \cdots$$

$$\mathbf{F} = \mathbf{I} + \mathbf{A}\Delta t + \mathbf{0} = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} + \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\Delta t = \begin{bmatrix}1 & \Delta t\\ 0 & 1\end{bmatrix}$$

$$\mathbf{x}_k = \begin{bmatrix}1 & \Delta t\\ 0 & 1\end{bmatrix}\mathbf{x}_{k-1}$$

Exact in two terms

- 등속 가정, $\dot{x}$ 를 v 로 치환한 1 차 미분방정식
- A 의 제곱이 0 이므로 이후 모든 거듭제곱도 0, 무한 급수 없이 정확한 해

Numerical issues

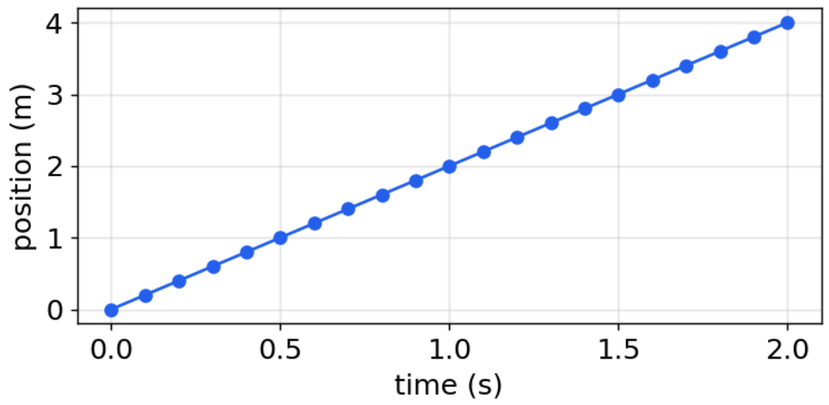
- SciPy expm() 은 Taylor 급수가 아닌 Padé 근사 사용
- 행렬 지수 계산법은 19 가지 이상, 모두 수치적 어려움, 큰 A 에서 주의
- Kalman filter 에서는 보통 Taylor 급수 앞 두 항만 사용

Constant Velocity with expm()

- 등속 모델 A 의 matrix exponential 을 scipy 의 expm() 으로 계산, 출력된 F 가 [[1, Δt], [0, 1]] 과 일치
- 구한 F 를 x_k = F x_k-1 로 20 번 적용, 위치가 일정한 기울기로 증가하는 등속 운동 확인
- **조건**: A = [[0, 1], [0, 0]], dt = 0.1 s, 초기 위치 0 m, 속도 2 m/s

```python
import numpy as np
from scipy.linalg import expm
dt = 0.1
A = np.array([[0, 1],
              [0, 0]])
F = expm(A*dt)
print(F)
# propagate x = [position, velocity]
x = np.array([0., 2.])
xs = [x]
for k in range(20):
    x = F @ x
    xs.append(x)
plt.plot(np.arange(21)*dt, np.array(xs)[:, 0], 'o-')
plots.set_labels(x='time (s)', y='position (m)')
```

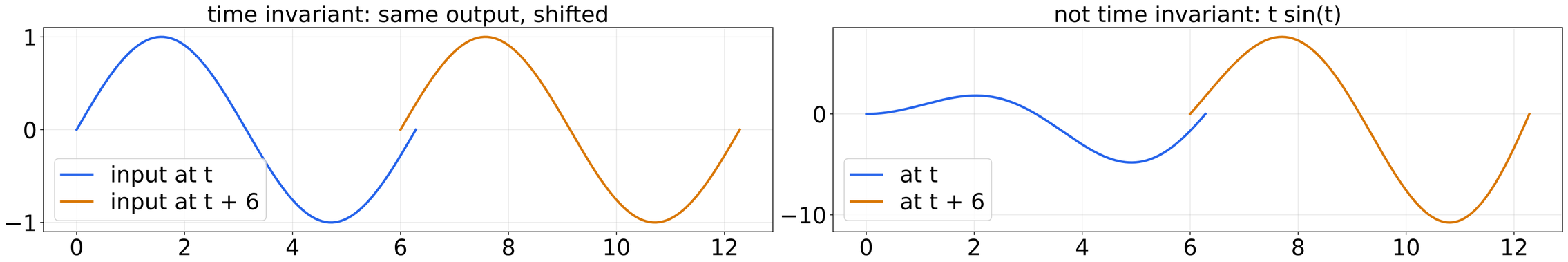
```
[[1.  0.1]
 [0.  1. ]]
```



Time Invariance

- **시간 불변**: 입력을 t 대신 t + Δt 에 넣으면 출력은 같은 f(x), 시간만 이동 (예: 오디오 앰프)
- **시변**: f(x) = t sin(t) 는 t 를 곱하므로 시간마다 다른 값, 연료를 소모해 무게가 변하는 항공기

$$g(t) = \dot{x} \quad (\text{time varying})$$
$$f(x) = \dot{x} \quad (\text{time invariant})$$



핵심: matrix exponential 은 시간 불변인 임의의 미분방정식 집합에 적용 가능

Separation of Variables

$$\frac{dx}{dt} = f(x)$$

$$\int_{x_0}^{x} \frac{1}{f(x)} dx = \int_{t_0}^{t} dt$$

$$F(x) = \int \frac{1}{f(x)} dx$$

$$F(x) - F(x_0) = t - t_0$$

$$x = F^{-1}[t - t_0 + F(x_0)]$$

Analytic solution

- 양변을 적분하려면 F 의 역함수가 필요
- 많은 f(x) 는 닫힌 해가 없거나 풀기 매우 어려움

State-space methods

- 실무 엔지니어는 근사 해를 위해 state-space 방법 사용
- 시변 시스템도 Δt 가 짧으면 거의 선형, 예: 1 초 동안 항공기 무게 감소는 무시 가능

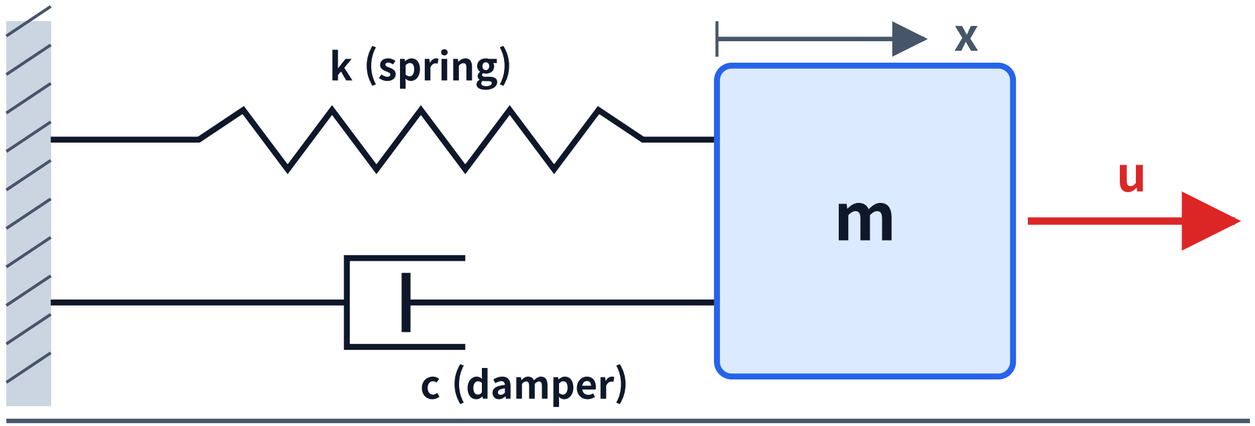
> **핵심**: 시변 시스템도 시간 간격이 짧으면 matrix exponential 로 충분히 정확한 결과

Example: Mass-Spring-Damper Model

$$m\ddot{x} + c\dot{x} + kx = u$$

$$x_1 = x, \quad x_2 = \dot{x}_1, \quad \dot{x}_2 = \ddot{x}$$

$$\dot{x}_2 = -\frac{c}{m}x_2 - \frac{k}{m}x_1 + \frac{1}{m}u$$



$$\begin{bmatrix}\dot{x}_1 \\ \dot{x}_2\end{bmatrix} = \begin{bmatrix}0 & 1 \\ -k/m & -c/m\end{bmatrix}\begin{bmatrix}x_1 \\ x_2\end{bmatrix} + \begin{bmatrix}0 \\ 1/m\end{bmatrix}u \qquad \mathbf{F} \approx \mathbf{I} + \mathbf{A}t = \begin{bmatrix}1 & t \\ -(k/m)t & 1-(c/m)t\end{bmatrix}$$

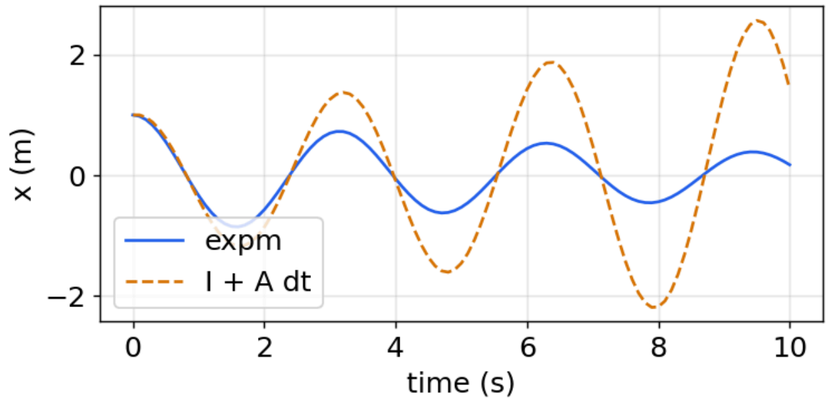
예: 자동차 서스펜션, m 질량, k 스프링 상수, c 감쇠, 앞 두 항으로 충분한지는 $(At)^2/2!$ 크기로 확인

Example: Mass-Spring-Damper Model

- 질량-스프링-감쇠 시스템의 F 를 급수 앞 두 항 I + AΔt 와 expm() 으로 각각 구해 같은 초기 상태를 전파
- 두 항 근사(점선)는 진폭이 커지고 정확한 해는 감쇠, 출력된 (AΔt)^2/2! 로 앞 두 항이 충분한지 직접 확인
- **조건**: m = 1 kg, k = 4 N/m, c = 0.2 N s/m, dt = 0.1 s, 초기 x = 1 m, 100 단계

```python
from scipy.linalg import expm
m, k, c = 1., 4., 0.2   # mass, spring constant, damping
dt = 0.1
A = np.array([[0., 1.], [-k/m, -c/m]])
F2 = np.eye(2) + A*dt   # first two terms
F = expm(A*dt)          # matrix exponential
print((A*dt) @ (A*dt) / 2)  # (A dt)^2 / 2!
x = x2 = np.array([1., 0.])
xs, x2s = [x], [x2]
for _ in range(100):
    x, x2 = F @ x, F2 @ x2
    xs.append(x); x2s.append(x2)
ts = np.arange(101)*dt
plt.plot(ts, np.array(xs)[:, 0], label='expm')
plt.plot(ts, np.array(x2s)[:, 0], '--', label='I + A dt')
plots.set_labels(x='time (s)', y='x (m)')
plt.legend(loc=3);
```

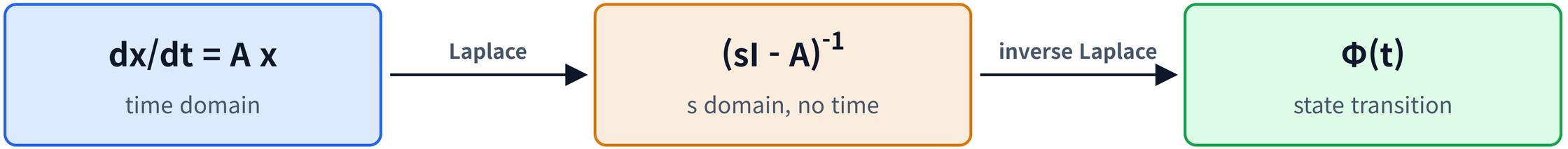
```
[[-0.02  -0.001]
 [ 0.004 -0.02 ]]
```



Linear Time Invariant Theory

$$\Phi(t) = \mathcal{L}^{-1}[(s\mathbf{I} - \mathbf{A})^{-1}]$$

- **LTI system theory**: inverse Laplace 변환으로 Φ 계산
- **Laplace 변환**: 신호를 시간이 없는 s 공간으로 변환, 위 식의 해를 구하는 일은 쉽지 않음
- **활용**: 어려운 문제의 Kalman filter 행렬 설계에 LTI 를 쓰는 문헌이 있음
- **예**: 등속 모델 A 에 적용하면 matrix exponential 과 같은 F



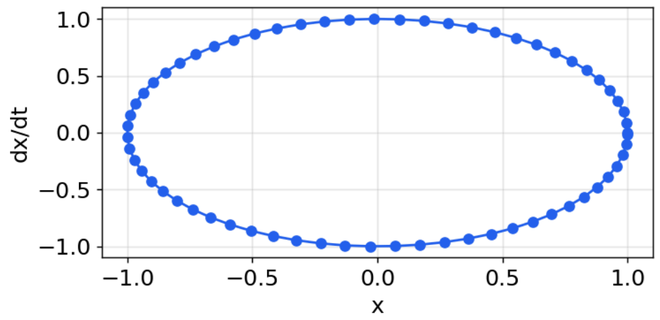
$$s\mathbf{I} - \mathbf{A} = \begin{bmatrix} s & -1 \\ 0 & s \end{bmatrix} \qquad (s\mathbf{I} - \mathbf{A})^{-1} = \begin{bmatrix} 1/s & 1/s^2 \\ 0 & 1/s \end{bmatrix} \qquad \Phi(t) = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

Numerical Solutions

- 필터가 커지면 해석 해가 번거로우므로 van Loan 방법으로 연속 모델 $\dot{x} = Ax + Gw$ 에서 F 와 Q 를 동시에 수치 계산
- 출력된 F 는 Δt 동안의 회전, 그래프는 F 를 반복 적용한 상태가 위상 평면에서 원을 그리는 모습
- **조건**: A = [[0, 1], [-1, 0]] (진동자), w 는 단위 white noise, G = [[0], [2]], dt = 0.1, 63 단계

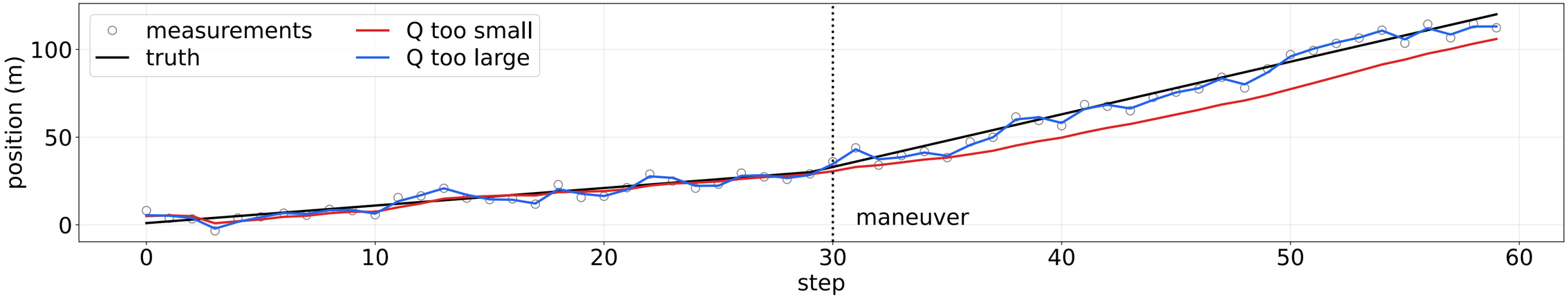
```python
from filterpy.common import van_loan_discretization
A = np.array([[0., 1.], [-1., 0.]])
G = np.array([[0.], [2.]]) # white noise scaling
F, Q = van_loan_discretization(A, G, dt=0.1)
print(F); print(Q)
# propagate x = [x, dx/dt] with F
x = np.array([1., 0.])
xs = [x]
for k in range(63):
    x = F @ x
    xs.append(x)
xs = np.array(xs)
plt.plot(xs[:, 0], xs[:, 1], 'o-')
plots.set_labels(x='x', y='dx/dt')
```

```
[[ 0.995  0.1  ]
 [-0.1    0.995]]
[[0.001 0.02 ]
 [0.02  0.399]]
```



Design of the Process Noise Matrix

- **어려움**: 신호 이론이 필요, 정보가 거의 없는 대상의 잡음을 모델링
- **예**: 던진 야구공의 회전, 실밥의 항력, 바람, 공기 밀도
- **Q 가 너무 작음**: 예측 모델을 과신해 실제 해에서 발산
- **Q 가 너무 큼**: 측정 잡음에 과하게 끌려가 최적보다 나쁜 성능



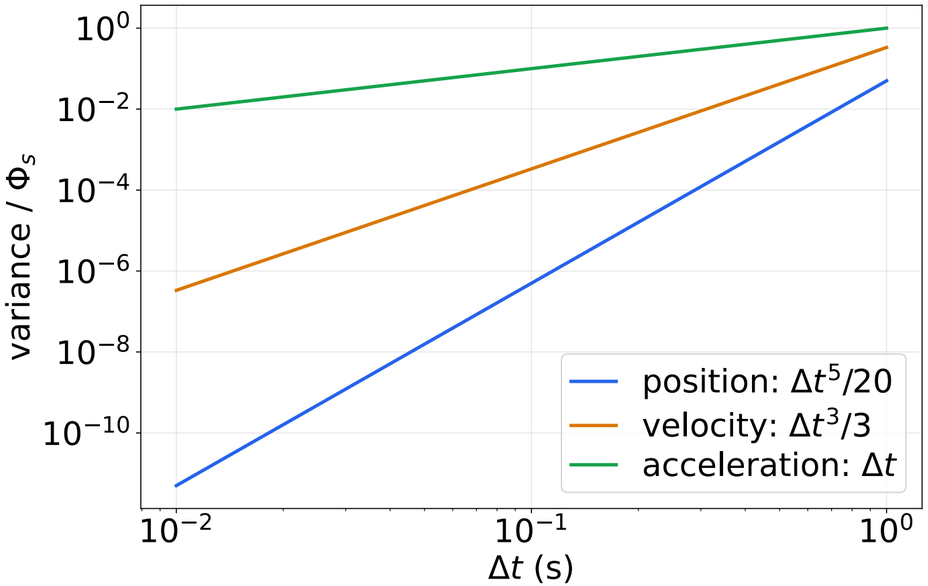
연속과 이산: 운동은 연속, 필터는 일정 간격으로 샘플링하므로 잡음 항의 이산 표현이 필요

Continuous White Noise Model

- **가정**: 구간마다 일정한 가속도가 zero-mean 연속 white noise w(t) 로 변화
- **이산화**: 연속 잡음을 F 로 투영하고 구간 [0, Δt] 에서 적분
- Φ_s: spectral density, 보통 모르므로 실험으로 조정하는 계수

$$\mathbf{Q} = \int_0^{\Delta t} \mathbf{F}(t)\mathbf{Q}_c\mathbf{F}^T(t)\,dt$$

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t & \Delta t^2/2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{Q}_c = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \Phi_s$$



핵심: Q_c 는 잡음 전력을 가속도 항에만 할당, 잡음이 가속도를 바꾸고 그 영향이 속도와 위치로 누적

Q for Position, Velocity, Acceleration

- 위치, 속도, 가속도 모델의 Q 를 SymPy 로 기호 적분, 가독성을 위해 Φs 를 행렬 밖으로 분리
- 결과의 위치 항은 Δt^5 에 비례, 가속도 항은 Δt 에 비례해 차수가 높은 상태일수록 빠르게 작아짐
- **조건**: F 는 3 x 3 등가속 모델, Q_c 는 가속도 항만 Φs

```python
import sympy
from sympy import Matrix, integrate, symbols

dt, phi = symbols(r'\Delta{t} \Phi_s')
F_k = Matrix([[1, dt, dt**2/2],
              [0,  1,      dt],
              [0,  0,       1]])
Q_c = Matrix([[0, 0, 0],
              [0, 0, 0],
              [0, 0, 1]])*phi

Q = integrate(F_k * Q_c * F_k.T, (dt, 0, dt))

# factor phi out of the matrix to make it more readable
Q = Q / phi
plots.show_matrix(Q, phi)
```

$$\begin{bmatrix} \frac{\Delta t^5}{20} & \frac{\Delta t^4}{8} & \frac{\Delta t^3}{6} \\ \frac{\Delta t^4}{8} & \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{6} & \frac{\Delta t^2}{2} & \Delta t \end{bmatrix} \Phi_s$$

0th and 1st Order Models

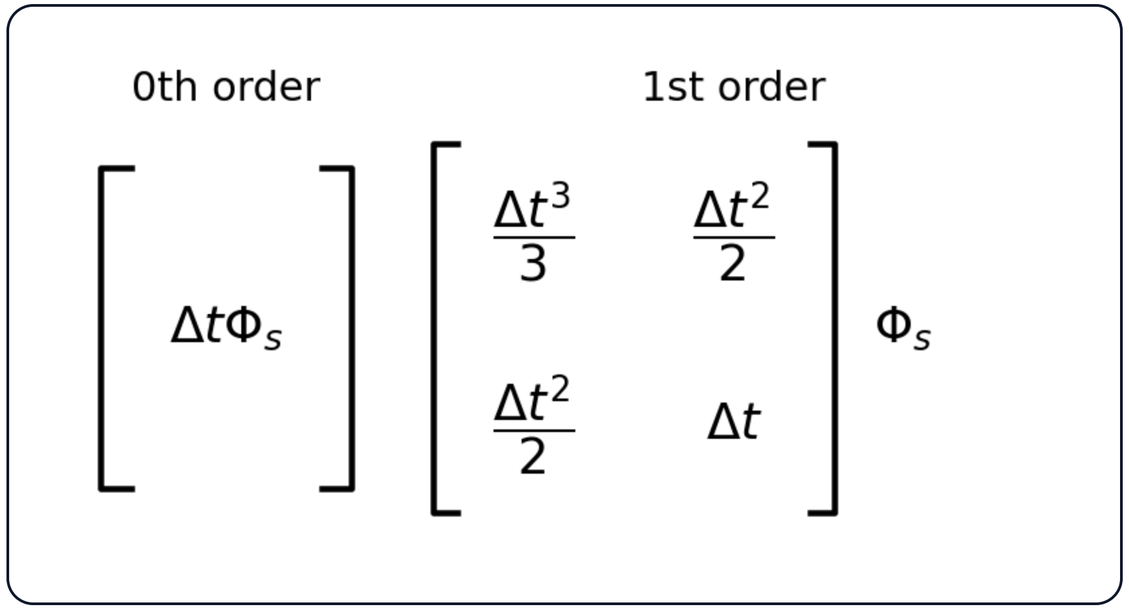
- 같은 적분을 0 차(위치만) 모델과 1 차(위치, 속도) 모델에 적용해 이산 process noise 계산
- 0 차는 ΔtΦs 하나, 1 차는 2 x 2 행렬이며 잡음은 각 모델의 최고차 항에 할당
- **조건**: 0 차: F = [1], Q_c = [Φs], 1 차: F = [[1, Δt], [0, 1]], Q_c 는 속도 항만 Φs

```python
from sympy import Matrix, integrate, symbols
dt, phi = symbols(r'\Delta{t} \Phi_s')

F_k = Matrix([[1]])
Q_c = Matrix([[phi]])
Q0 = integrate(F_k*Q_c*F_k.T, (dt, 0, dt))

F_k = Matrix([[1, dt],
              [0, 1]])
Q_c = Matrix([[0, 0],
              [0, 1]]) * phi
Q1 = integrate(F_k * Q_c * F_k.T, (dt, 0, dt)) / phi

plt.subplot(1, 3, 1)
plots.show_matrix(Q0, title='0th order')
plt.subplot(1, 3, (2, 3))
plots.show_matrix(Q1, phi, title='1st order')
```

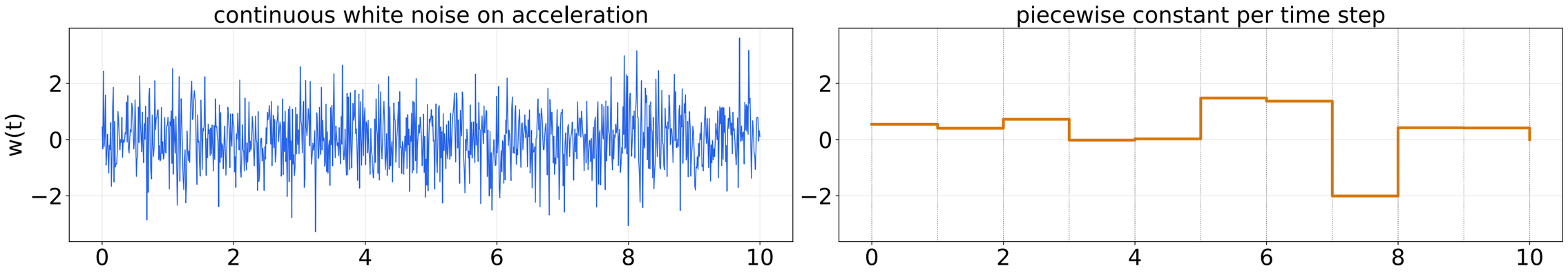


Piecewise White Noise Model

- **가정**: 최고차 항(예: 가속도)은 각 구간 동안 일정, 구간마다 다르고 서로 상관 없음
- **차이**: 구간 경계에서 불연속 점프, 연속적으로 변하는 잡음을 가정한 모델과 구별
- Γ: noise gain, 한 구간에서 속도 변화 w(t)Δt, 위치 변화 w(t)Δt²/2

$$f(\mathbf{x}) = \mathbf{F}\mathbf{x} + \Gamma w$$

$$\Gamma = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix}, \quad \mathbf{Q} = E[\Gamma w(t) w(t) \Gamma^T] = \Gamma \sigma_v^2 \Gamma^T$$



Q for 1st and 2nd Order Models

- 구간 상수 잡음 모델의 Q = Γ σv² Γ^T 를 1 차와 2 차 모델에 대해 SymPy 로 계산
- 2 차 모델은 잡음을 discrete time Wiener process 로 가정해 Γ = [Δt²/2, Δt, 1]^T
- 이 모델이 참이라는 근거는 없으며 편리하고 결과가 좋음, 잡음을 jerk 에 적용하는 모델도 가능
- **조건**: σv² 는 행렬 밖으로 분리, 왼쪽 1 차 (2 x 2), 오른쪽 2 차 (3 x 3)

```python
from sympy import Matrix, symbols
dt, var = symbols(r'\Delta{t} sigma^2_v')

v = Matrix([[dt**2 / 2], [dt]])
Q1 = v * var * v.T
Q1 = Q1 / var  # factor variance out of the matrix

v = Matrix([[dt**2 / 2], [dt], [1]])
Q2 = v * var * v.T
Q2 = Q2 / var

plt.subplot(1, 2, 1)
plots.show_matrix(Q1, var, title='1st order')
plt.subplot(1, 2, 2)
plots.show_matrix(Q2, var, title='2nd order')
```

1st order

$$\begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix} \sigma_v^2$$

2nd order

$$\begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 & \Delta t \\ \frac{\Delta t^2}{2} & \Delta t & 1 \end{bmatrix} \sigma_v^2$$

Choosing a Noise Model

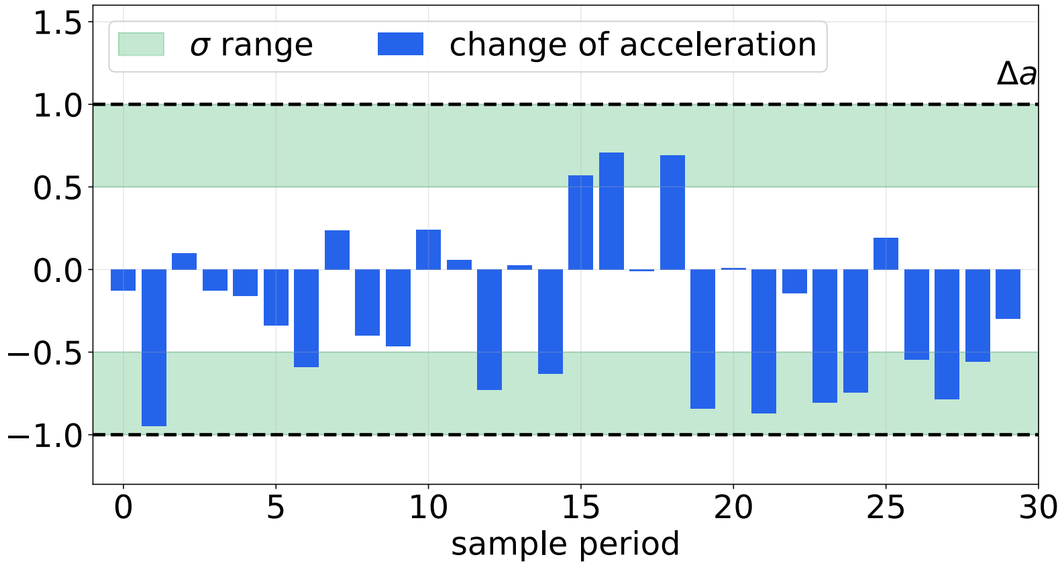
두 모델 모두 실제 물체의 움직임에 대한 근사, 어느 쪽이 더 옳다고 할 수 없으며 경험과 실험으로 선택

Continuous white noise

- **잡음 표현**: spectral density Φ_s, 직관적이지 않음
- **장점**: 구간 전체에 적분하므로 샘플 간격이 변해도 쉽게 처리

Piecewise white noise

- **잡음 표현**: σ^2, 운동과 예상 오차로 설명 가능
- **장점**: 값의 의미가 분명해 설정이 쉬움



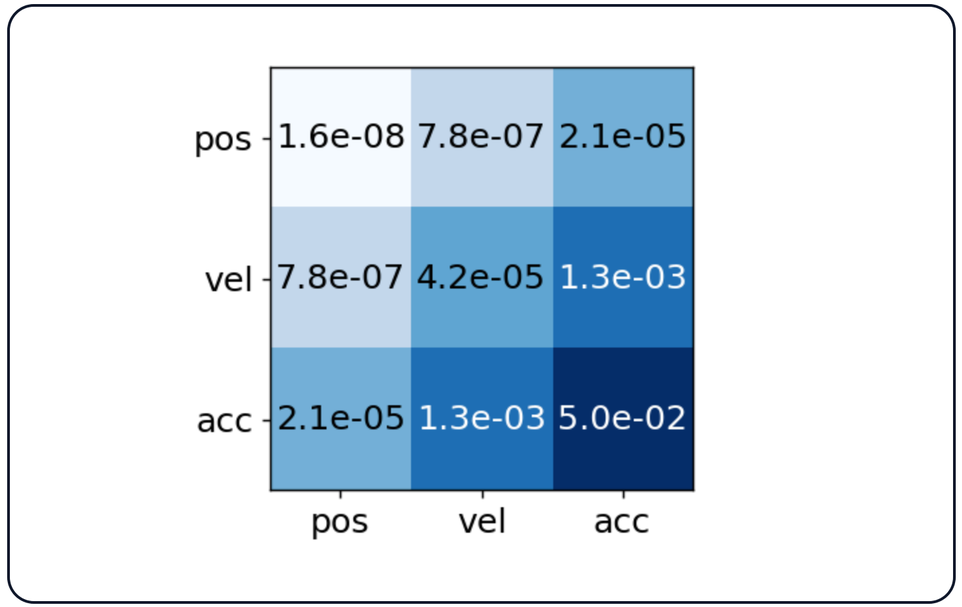
경험 법칙: $\frac{1}{2}\Delta a \leq \sigma \leq \Delta a$, Δa 는 샘플 사이 가속도 최대 변화량, 이후 데이터로 시뮬레이션해 결정

Simplification of Q

- 짧은 Δt 에서 연속 white noise 모델의 Q 를 계산, 가속도 항 0.05 에 비해 위치 항 2e-08 등 대부분이 매우 작음
- Q 를 쓰는 식은 $\bar{P} = FPF^T + Q$ 뿐이므로 P 에 비해 작은 원소는 결과에 거의 기여하지 않음
- **조건**: dt = 0.05 s, spectral density 1, 3 x 3 (위치, 속도, 가속도)

```python
dt = 0.05
# continuous model, spectral density 1
Q = np.array([[dt**5/20, dt**4/8, dt**3/6],
              [dt**4/8,  dt**3/3, dt**2/2],
              [dt**3/6,  dt**2/2, dt     ]])

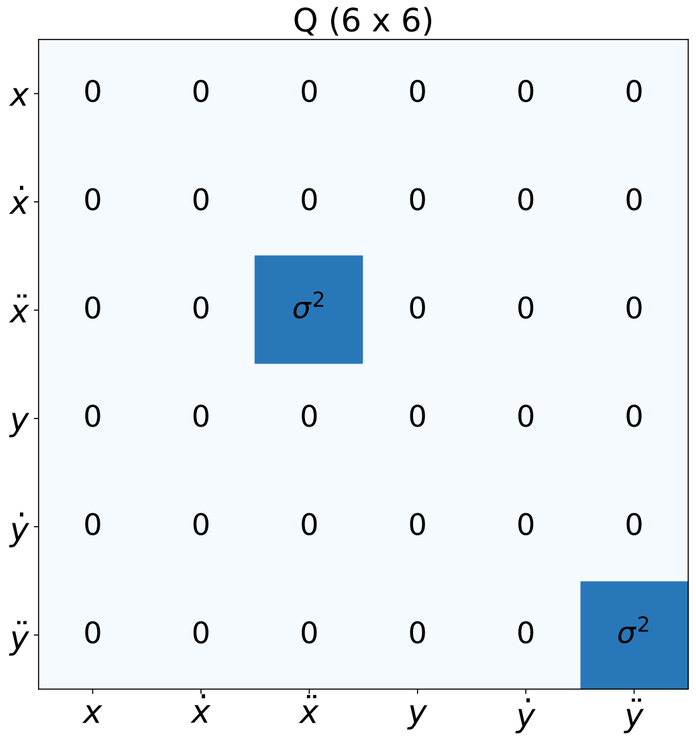
plt.imshow(np.log10(Q), cmap='Blues')
for (i, j), q in np.ndenumerate(Q):
    plt.text(j, i, '{:.1e}'.format(q), ha='center', va='center',
             color='k' if i + j < 3 else 'w')
names = ['pos', 'vel', 'acc']
plt.xticks(range(3), names);
plt.yticks(range(3), names);
plt.grid(False);
```



Lower Right Term Only

$$\mathbf{Q} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \sigma^2 \end{bmatrix}$$

- **근사**: 오른쪽 아래 항만 남기고 나머지를 0 으로, 정확하지 않지만 흔히 유용한 근사
- **주의**: 중요한 응용이라면 다양한 상황에서 동작하는지 충분한 검증 필요
- **의미**: 오른쪽 아래 항은 변수마다 가장 빠르게 변하는 항
- **다변수**: $\mathbf{x} = [x\ \dot{x}\ \ddot{x}\ y\ \dot{y}\ \ddot{y}]^T$ 이면 Q 는 6 x 6, $\ddot{x}$ 와 $\ddot{y}$ 원소 모두 0 이 아님



Joseph Form

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}$$

$$\mathbf{P} = (\mathbf{I} - \mathbf{KH})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{KH})^T + \mathbf{KRK}^T$$

Nonsymmetric P

- 부동소수점 오차로 (I - KH) 뺄셈이 비대칭 행렬을 만듦
- 공분산은 대칭이어야 하므로 필터 발산, NumPy 검사에서 예외 발생

Symmetrize

$$\mathbf{P} = (\mathbf{P} + \mathbf{P}^T)/2$$

- $\sigma_{ij} = \sigma_{ji}$ 이므로 안전, 어긋난 두 값의 오차를 평균

Joseph form

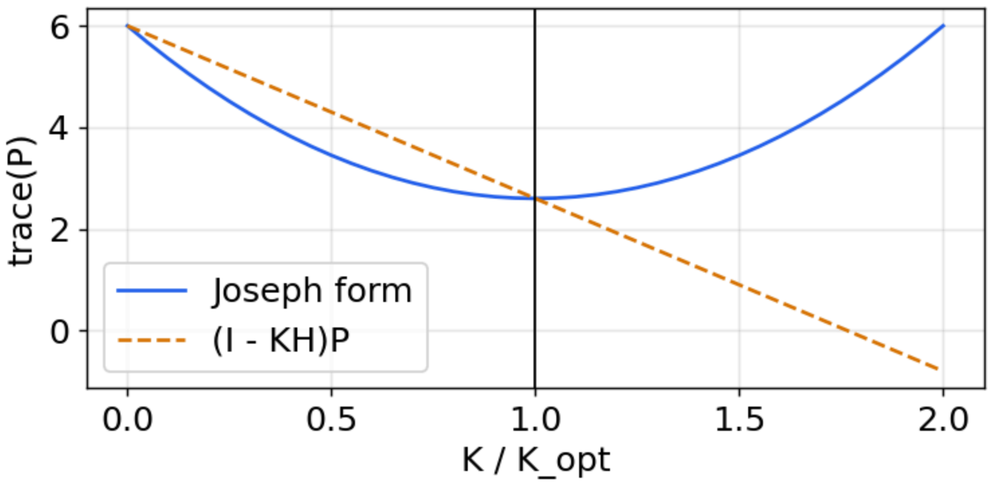
- 두 항 모두 ABA^T 형태라 대칭 유지
- 최적이 아닌 gain 에서도 성립, 실제 구현에서 사용

핵심: 단순식이 틀린 것은 아니지만 수치 안정성과 임의 gain 에서의 정확성 때문에 Joseph form 사용

Joseph Form with Any Gain

- 최적 gain 에 배율 s 를 곱한 임의의 gain 으로 두 식의 trace 를 비교, 두 값은 s = 1 (K = K_opt) 에서만 일치
- Joseph form 은 s = 1 에서 최소, 단순식은 큰 gain 에서 음수가 되어 공분산으로 성립하지 않음
- **조건**: P = [[4, 1], [1, 2]], H = [1, 0], R = 1, K = s K_opt, s 는 0 ~ 2

```python
P = np.array([[4., 1.],
              [1., 2.]])     # prior covariance
H, R = np.array([[1., 0.]]), np.array([[1.]])
K_opt = P @ H.T @ np.linalg.inv(H @ P @ H.T + R)
scales = np.linspace(0, 2, 41)
joseph, simple = [], []
for s in scales:
    K = s * K_opt           # suboptimal gain when s != 1
    A = np.eye(2) - K @ H
    joseph.append(np.trace(A @ P @ A.T + K @ R @ K.T))
    simple.append(np.trace(A @ P))
plt.plot(scales, joseph, label='Joseph form')
plt.plot(scales, simple, '--', label='(I - KH)P')
plt.axvline(1, color='k', lw=1)
plots.set_labels(x='K / K_opt', y='trace(P)')
plt.legend();
```



Derivation: Definitions

$$\mathbf{x}_{k+1} = \mathbf{F}_k \mathbf{x}_k + \mathbf{w}_k$$

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

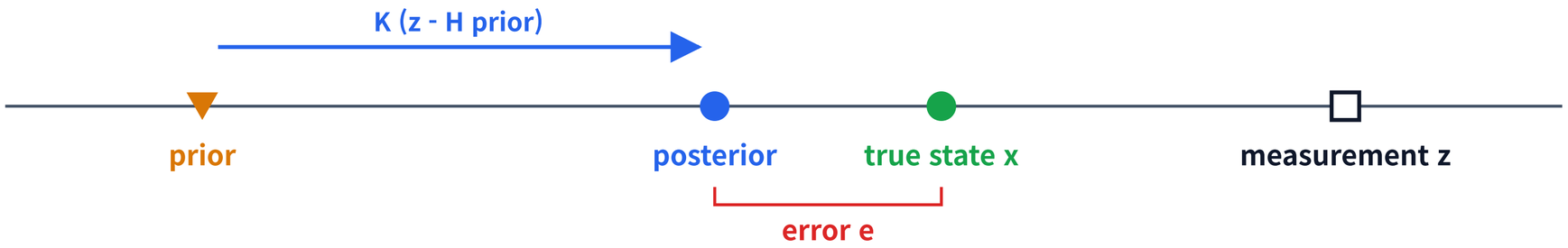
$$\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$$

- **기호**: x 는 참 상태, $\hat{x}$ 는 추정 상태(posterior), $\bar{x}$ 는 prior
- **잡음**: 어떤 시스템도 모델을 완벽히 따르지 않아 w, 측정은 센서 오차 v

$$\mathbf{P} = E[\mathbf{e}\mathbf{e}^T] = E[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^T]$$

$$\hat{\mathbf{x}} = \bar{\mathbf{x}} + \mathbf{K}(\mathbf{z} - \mathbf{H}\bar{\mathbf{x}})$$

- **e**: 참 상태와 추정 상태의 차이, 계산할 수는 없는 정의
- **K**: Kalman gain 이 아닌 0 ~ 1 사이의 임의 혼합 계수, 선형 Kalman filter 에 한정하지 않음



Derivation: Expanding the Error

$$
\begin{aligned}
(\mathbf{x} - \hat{\mathbf{x}}) &= \mathbf{x} - (\bar{\mathbf{x}} + \mathbf{K}(\mathbf{z} - \mathbf{H}\bar{\mathbf{x}})) \\
&= \mathbf{x} - (\bar{\mathbf{x}} + \mathbf{K}(\mathbf{H}\mathbf{x} + \mathbf{v} - \mathbf{H}\bar{\mathbf{x}})) \\
&= (\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}(\mathbf{H}\mathbf{x} + \mathbf{v} - \mathbf{H}\bar{\mathbf{x}}) \\
&= (\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}\mathbf{H}(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}\mathbf{v} \\
&= (\mathbf{I} - \mathbf{K}\mathbf{H})(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}\mathbf{v}
\end{aligned}
$$

- **기댓값**: $E[(x - \bar{x})(x - \bar{x})^T] = \bar{P}$, $E[vv^T] = R$, 측정 잡음과 prior 오차는 서로 무관

$$
\begin{aligned}
\mathbf{P} &= E\left[[(\mathbf{I} - \mathbf{K}\mathbf{H})(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}\mathbf{v}][(\mathbf{I} - \mathbf{K}\mathbf{H})(\mathbf{x} - \bar{\mathbf{x}}) - \mathbf{K}\mathbf{v}]^T\right] \\
&= (\mathbf{I} - \mathbf{K}\mathbf{H})\bar{\mathbf{P}}(\mathbf{I} - \mathbf{K}\mathbf{H})^T + \mathbf{K}\mathbf{R}\mathbf{K}^T
\end{aligned}
$$

> **핵심**: 최적 gain 뿐 아니라 임의의 K 에서 성립, 실제 gain 은 비선형성과 부동소수점 오차로 최적이 아니므로 더 안정

Derivation: Optimal Gain

$$\mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^T(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^T + \mathbf{R})^{-1}$$

$$
\begin{aligned}
&(I - KH)\bar{P}(I - KH)^T + KRK^T \\
&= \bar{P} - KH\bar{P} - \bar{P}H^TK^T + K(H\bar{P}H^T + R)K^T \\
&= \bar{P} - KH\bar{P} - \bar{P}H^TK^T + \bar{P}H^T(H\bar{P}H^T + R)^{-1}(H\bar{P}H^T + R)K^T \\
&= \bar{P} - KH\bar{P} - \bar{P}H^TK^T + \bar{P}H^TK^T \\
&= (I - KH)\bar{P}
\end{aligned}
$$

Optimal gain

- 최적 gain 이면 두 식 모두 수학적으로 정확
- Joseph form 은 최적이 아닌 gain 에서도 정확하고 더 안정

Divergence

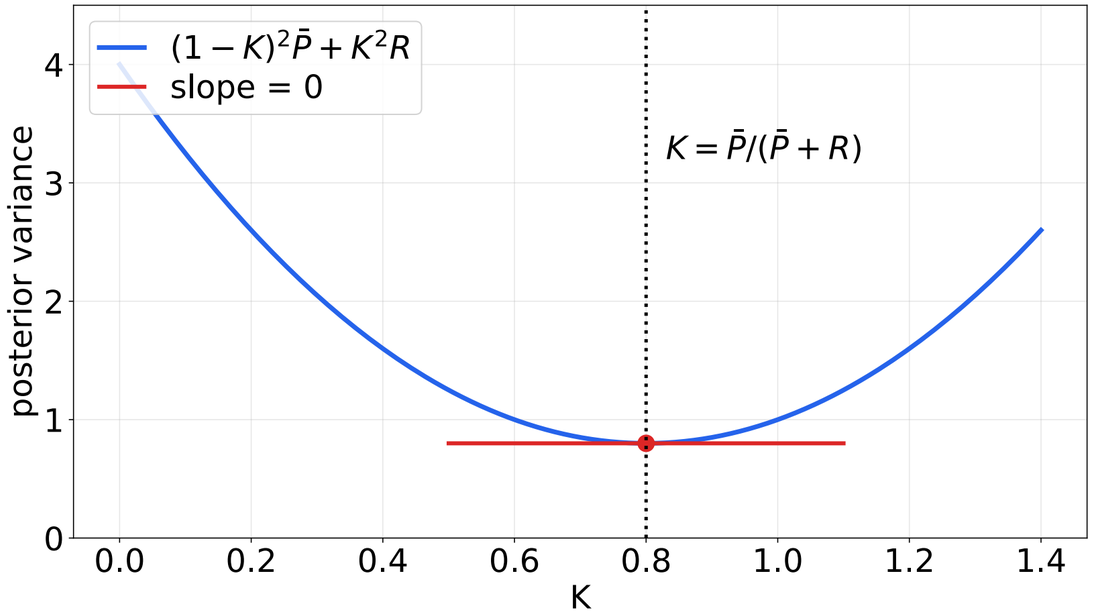
- 수백, 수천 epoch 실행하면 여전히 발산 가능
- 피해가 없는 문제면 P 재설정 (비대각 0, 1 보다 약간 큰 상수 곱)

Minimizing the Trace of P

- **관점**: Bayes 식이 아닌 미분으로 오차를 최소화하는 유도
- **목표**: P 의 대각은 상태 원소별 분산, 대각의 합 trace(P) 를 K 로 미분해 0 이 되는 K 탐색
- **trace 미분 공식**: AB 는 정방, C 는 대칭

$$\frac{d\,trace(\mathbf{AB})}{d\mathbf{A}} = \mathbf{B}^T \qquad \frac{d\,trace(\mathbf{ACA}^T)}{d\mathbf{A}} = 2\mathbf{AC}$$

$$\frac{d}{dx} f(x) = 0$$



예: 스칼라에서 $P = (1-K)^2\bar{P} + K^2R$ 의 최소점이 $K = \bar{P}/(\bar{P}+R)$, 1 차원 Kalman gain 과 동일

Solving for K

Term	$d\,trace(\cdot)/dK$	Reason
$P = \bar{P}$	0	K 를 포함하지 않음
$-KH\bar{P}$	$-(H\bar{P})^T$	trace(AB) 공식
$-\bar{P}H^TK^T$	$-(H\bar{P})^T$	두 번째 항의 전치, trace 는 전치와 같음
$K(H\bar{P}H^T + R)K^T$	$2K(H\bar{P}H^T + R)$	trace(ACA^T) 공식

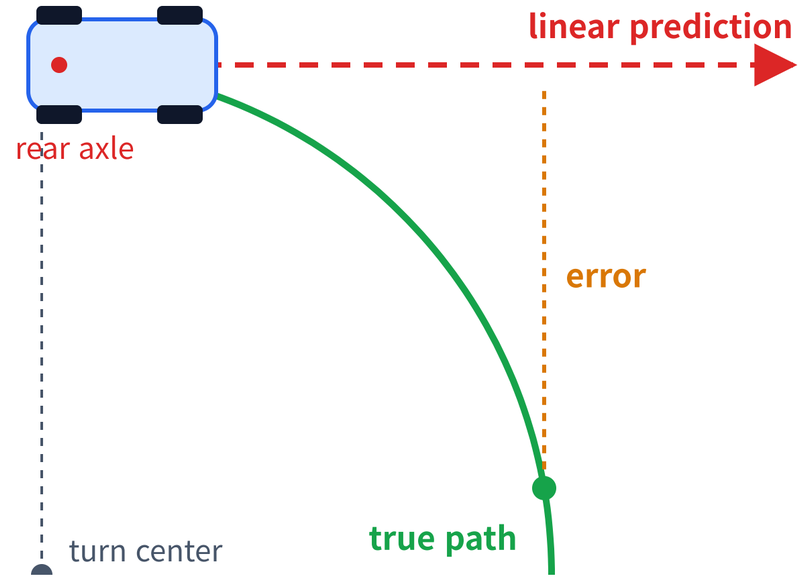
$$\frac{d\,trace(\mathbf{P})}{d\mathbf{K}} = -2(\mathbf{H}\bar{\mathbf{P}})^T + 2\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^T + \mathbf{R}) = 0$$

$$\mathbf{K}(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^T + \mathbf{R}) = (\mathbf{H}\bar{\mathbf{P}})^T = \bar{\mathbf{P}}\mathbf{H}^T \quad \Longrightarrow \quad \mathbf{K} = \bar{\mathbf{P}}\mathbf{H}^T(\mathbf{H}\bar{\mathbf{P}}\mathbf{H}^T + \mathbf{R})^{-1}$$

참고: trace 최소화가 전체 오차 최소화인 이유는 생략, 엄밀한 증명은 아니지만 Kalman gain 식을 얻는 과정

Numeric Integration of Differential Equations

- **지금까지**: state-space, Laplace 변환, van Loan 방법은 선형 상미분방정식(ODE)에 적합
- **한계**: 비선형 방정식에는 잘 맞지 않음
- **예**: 자동차는 앞바퀴를 돌려 뒤 차축을 중심으로 회전, 경로가 계속 휘어 선형 예측은 틀린 값
- **필요**: Δt 에 비해 변화가 작으면 선형도 쓸 만하지만 비선형 Kalman filter 에서는 드묾, ODE 의 수치 적분 필요



> **핵심**: 몇 가지 간단한 수치 적분 기법으로 대부분의 문제 해결 가능

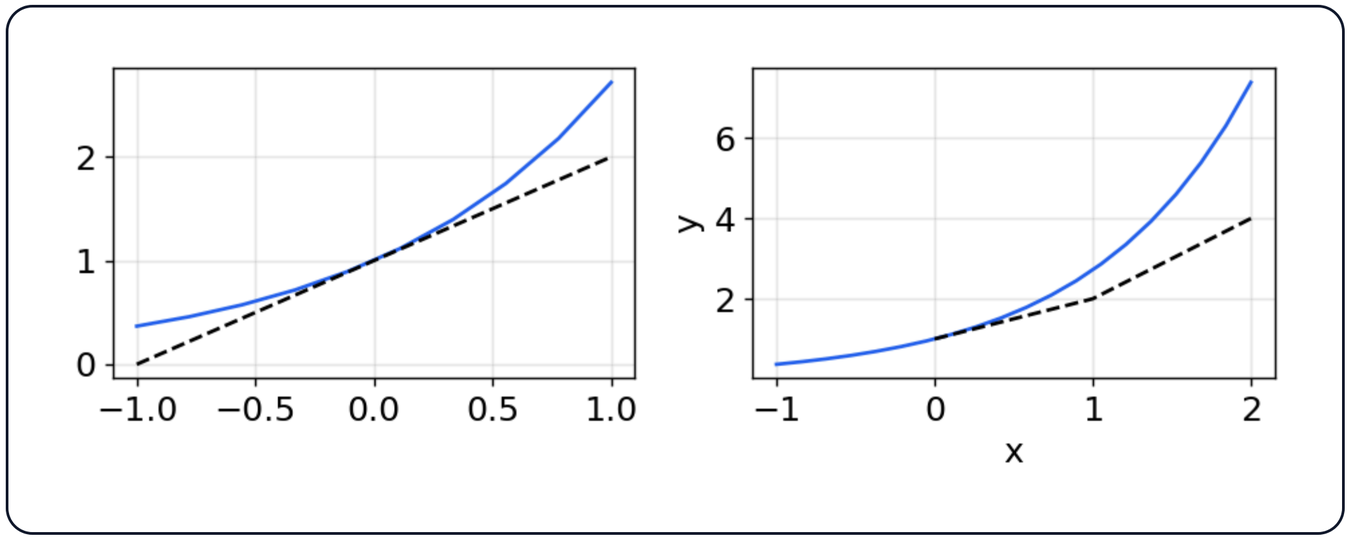
Slope at the Initial Value

- 초기값 문제 y' = y, y(0) = 1 에서 아는 것은 기울기(미분)와 초기값뿐, 정확한 해는 y = e^t
- 왼쪽: t = 0 의 기울기로 그은 직선, 0 근처에서는 곡선과 가깝지만 t = 1 에서는 멀어짐
- 오른쪽: 간격 1 로 t = 1 에서 2, 그 기울기 2 로 t = 2 에서 4 를 추정, 오차가 빠르게 커짐
- **조건**: step 1, 점선은 Euler 추정, 실선은 e^t

```python
import numpy as np
import matplotlib.pyplot as plt

plt.subplot(1, 2, 1)
t = np.linspace(-1, 1, 10)
plt.plot(t, np.exp(t))
t = np.linspace(-1, 1, 2)
plt.plot(t,t+1, ls='--', c='k');

plt.subplot(1, 2, 2)
t = np.linspace(-1, 2, 20)
plt.plot(t, np.exp(t))
plt.plot([1, 2, 4], ls='--', c='k')
plots.set_labels(x='x', y='y');
```

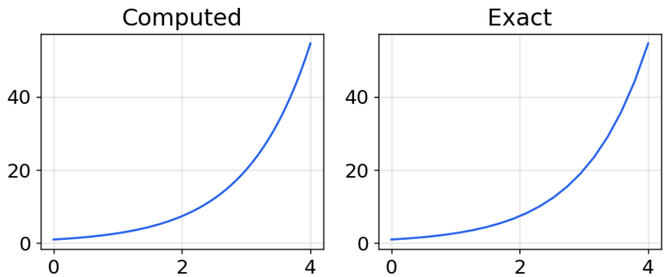


Euler in Code

- 기울기에 step 을 곱해 더하는 Euler 법을 함수로 구현, step 1 에서 2.0, 4.0 으로 손 계산과 일치
- step 0.00001 의 오차는 작지만 40 만 번 반복이 필요, 너무 느려 실제로는 더 정교한 방법 사용
- **조건**: y' = y, y(0) = 1, t = 0 ~ 4, step 1 과 0.00001

```python
def euler(t, tmax, y, dx, step=1.):
    ys = []
    while t < tmax:
        y = y + step*dx(t, y)
        ys.append(y)
        t +=step
    return ys
def dx(t, y): return y
print(euler(0, 1, 1, dx)[-1], euler(0, 2, 1, dx)[-1])
ys = euler(0, 4, 1, dx, step=0.00001)
plt.subplot(1,2,1); plt.title('Computed')
plt.plot(np.linspace(0, 4, len(ys)),ys)
plt.subplot(1,2,2); plt.title('Exact')
t = np.linspace(0, 4, 20)
plt.plot(t, np.exp(t));
print('exact answer=', np.exp(4))
print('euler answer=', ys[-1])
print('difference =', np.exp(4) - ys[-1])
print('iterations =', len(ys))
```

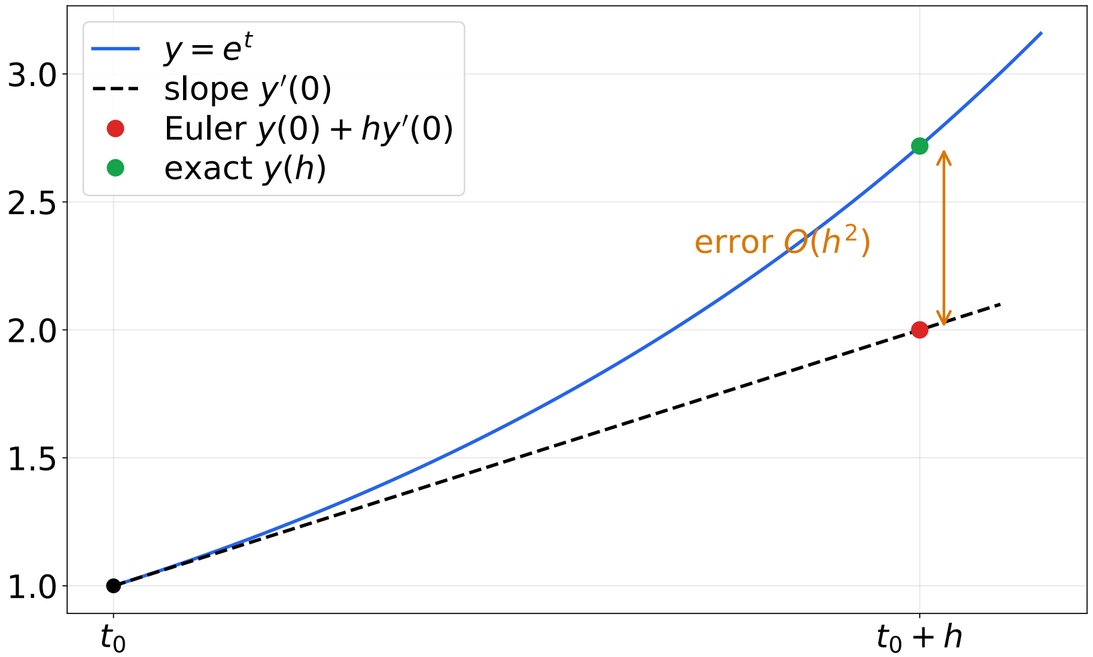
```
2.0 4.0
exact answer= 54.598150033144236
euler answer= 54.59705808834125
difference = 0.0010919448029866885
iterations = 400000
```



Taylor Expansion

$$y(t_0 + h) = y(t_0) + hy'(t_0) + \frac{1}{2!}h^2y''(t_0) + \frac{1}{3!}h^3y'''(t_0) + O(h^4)$$

- **Euler 법**: Taylor 전개의 처음 두 항 $y(t_0) + hy'(t_0)$ 만 사용
- $O(h^4)$: 유한 전개로 생기는 오차, 무한 전개면 정확한 해
- **오차**: 이후 항은 앞 항보다 작아 추정이 정답에서 크게 벗어나지 않음, h 가 크면 오차 증가
- **관계**: Euler 법은 Runge Kutta 의 가장 단순한 형태



RK4 Method

$$\ddot{y} = \frac{d}{dt}\dot{y} = \frac{d}{dt}f(y,t)$$

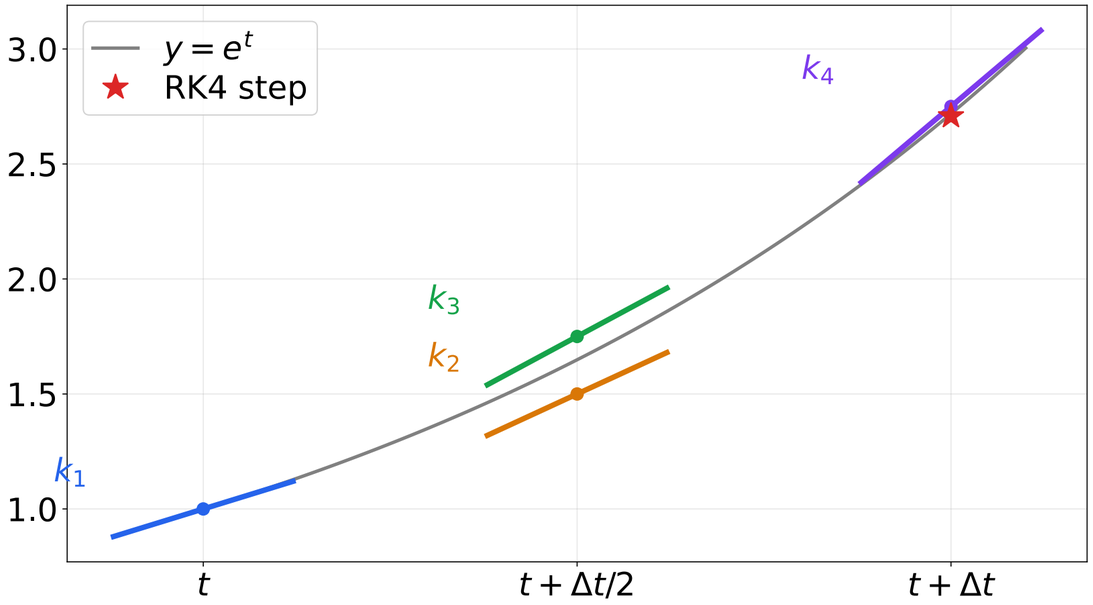
$$y(t+\Delta t) = y(t) + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) + O(\Delta t^4)$$

$$k_1 = f(y,t)\Delta t$$

$$k_2 = f(y + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)\Delta t$$

$$k_3 = f(y + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)\Delta t$$

$$k_4 = f(y + k_3, t + \Delta t)\Delta t$$



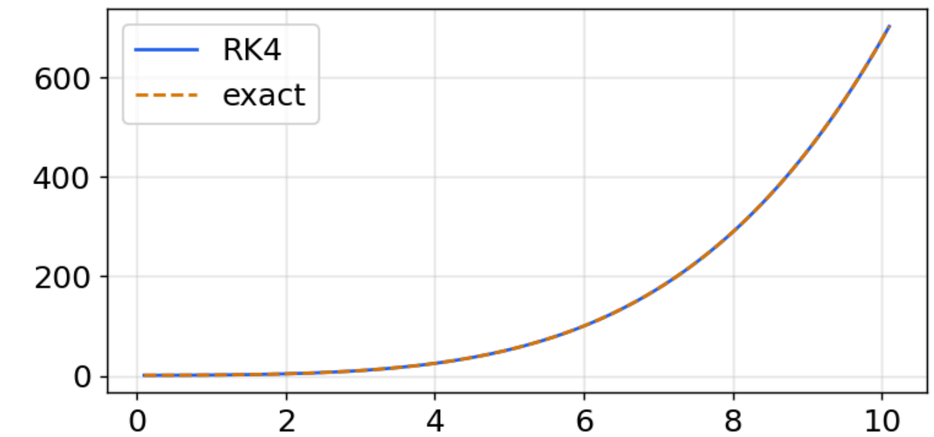
Runge Kutta: 속도, 정밀도, 안정성의 균형이 좋은 주력 기법, 네 기울기를 1 : 2 : 2 : 1 로 가중 평균

RK4 in Code

- RK4 함수로 $\dot{y} = t\,y^{1/2}$ 를 t = 0 ~ 10 까지 0.1 간격으로 적분하고 정확한 해와 비교
- 두 곡선이 겹치고 최대 오차는 0.00005 수준, Euler 법보다 훨씬 적은 반복으로 높은 정밀도
- **조건**: t0 = 0, y0 = 1, dt = 0.1, 정확한 해 y = (t^2 + 4)^2 / 16

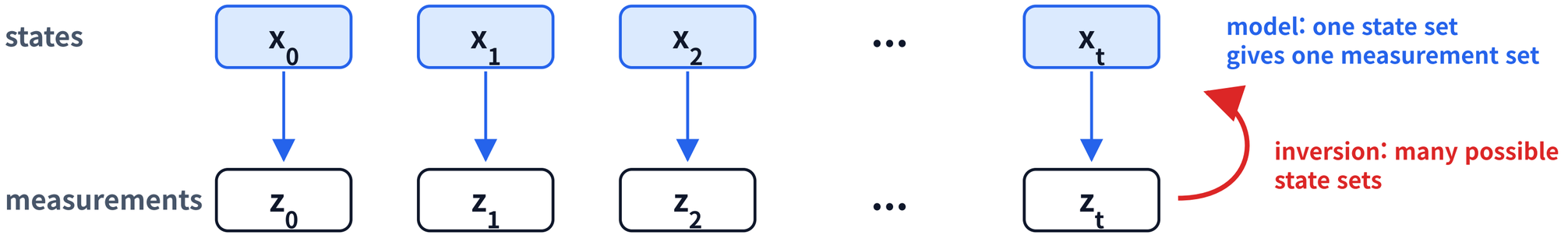
```python
def runge_kutta4(y, x, dx, f):
    """computes 4th order Runge-Kutta for dy/dx"""
    k1 = dx * f(y, x)
    k2 = dx * f(y + 0.5*k1, x + 0.5*dx)
    k3 = dx * f(y + 0.5*k2, x + 0.5*dx)
    k4 = dx * f(y + k3, x + dx)
    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.
t, y, dt = 0., 1., .1
ys, ts = [], []
def func(y,t): return t*np.sqrt(y)
while t <= 10:
    y = runge_kutta4(y, t, dt, func)
    t += dt
    ys.append(y); ts.append(t)
exact = [(t**2 + 4)**2 / 16. for t in ts]
plt.plot(ts, ys, label='RK4')
plt.plot(ts, exact, '--', label='exact')
plt.legend()
error = np.array(exact) - np.array(ys)
print("max error {:.5f}".format(max(error)))
```

```
max error 0.00005
```



Statistical Inversion

- **상태**: 시각 t 의 위치, 속도 등, 예: $\mathbf{x}_t = [x_t \ \dot{x}_t]^T$
- **측정**: 상태가 측정을 결정, 측정이 바뀐다고 상태가 바뀌지는 않음
- **역문제**: 측정 $z_{0:t}$ 로부터 상태 $x_{0:t}$ 를 계산, 출력에서 입력을 구함
- **어려움**: 한 측정 집합을 만들 수 있는 상태 집합이 여러 개, 해가 유일하지 않음

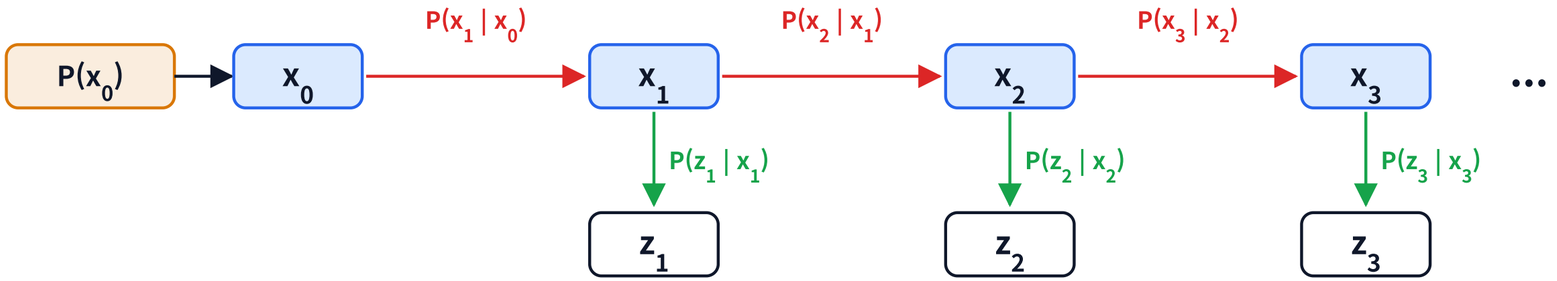


$$P(x \mid z) = \frac{P(z \mid x)P(x)}{P(z)} \qquad P(x_{0:t} \mid z_{0:t}) = \frac{P(z_{0:t} \mid x_{0:t})P(x_{0:t})}{P(z_{0:t})}$$

Recursive Bayesian Filter

$$x_k \sim P(x_k \mid x_{k-1}) \qquad z_k \sim P(z_k \mid x_k) \qquad x_0 \sim P(x_0)$$

- **문제**: 측정 z_{t+1} 마다 0 ~ t + 1 전체 재계산은 불가능, 현재 시점 분포만 계산
- **Markov**: 현재 상태는 이전 상태와 전이 확률에만 의존, 1 초 뒤 위치는 1 분 전 경로와 무관
- **반복**: Bayes 정리로 posterior, 운동 모델로 다음 prior, predictor-corrector 구조
- **Kalman filter**: 선형 모델과 zero-mean 무상관 Gaussian 잡음이면 최소제곱 의미에서 최적



> **관점**: 무작위성은 정보의 부족, 모르는 값을 확률 분포로 기술하고 Bayes 정리로 통계적 추론 문제를 해결

Kalman Filter as a g-h Filter

$$\mu_x = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2} = \frac{ya + xb}{a+b}$$

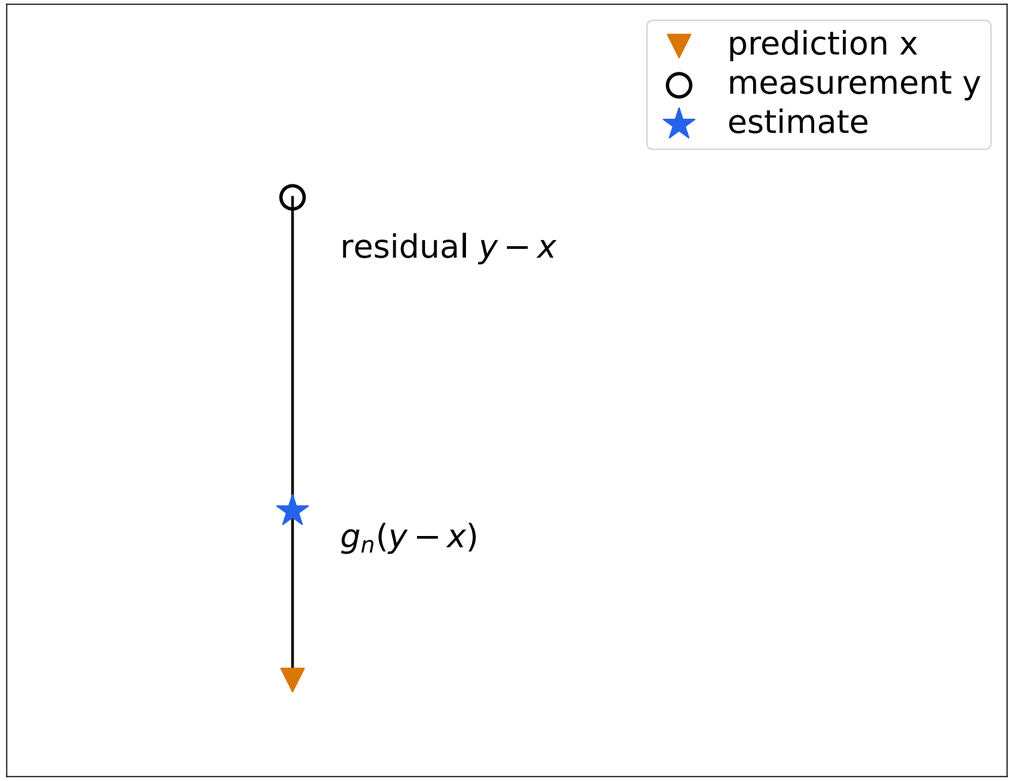
$$\mu_x = x + \frac{-x(a+b) + xb + ya}{a+b} = x + \frac{a}{a+b}(y-x)$$

$$\frac{a}{a+b} = \frac{1}{1+\frac{b}{a}} = \frac{1}{b}\cdot\frac{1}{\frac{1}{b}+\frac{1}{a}} = \frac{\sigma_x^2}{b}, \quad \sigma_x^2 = \frac{1}{\frac{1}{a}+\frac{1}{b}}$$

$$\mu_x = x + \frac{\sigma_x^2}{b}(y-x) = x + g_n(y-x)$$

- **기호**: x 는 예측(분산 a), y 는 측정(분산 b), 1 차원에서 대수로 증명

$$g_n = \frac{\sigma_x^2}{\sigma_y^2} \qquad h_n = \frac{COV(x,\dot{x})}{\sigma_y^2}$$



Gain g Varies with Time

- 1 차원 Kalman filter 를 반복하며 매 단계 Kalman gain K 와 g_n = (추정 분산) / (측정 분산) 을 함께 기록
- 두 값이 모든 단계에서 겹침, g 는 상수가 아니라 분산이 변하는 동안 단계마다 바뀌다가 수렴
- g, h 는 측정과 예측의 분산, 공분산으로 결정, 두 입력의 품질로 정한 비율만큼 둘 사이의 점을 선택
- **조건**: 초기 분산 500, process 분산 Q = 1, 측정 분산 R = 10, 30 단계

```python
P = 500.             # initial variance
Q, R = 1., 10.       # process and measurement variance
gs, Ks = [], []
for k in range(30):
    P = P + Q            # predict
    K = P / (P + R)      # Kalman gain
    P = (1 - K) * P      # posterior variance
    Ks.append(K)
    gs.append(P / R)     # g_n = sigma_x^2 / sigma_y^2

plt.plot(Ks, 'o', ms=10, label='Kalman gain K')
plt.plot(gs, lw=3, label='g_n = P / R')
plots.set_labels(x='step', y='gain')
plt.legend();
```

